After making that change, reloading your config, and tailing your log file, run SELECT pg_sleep(1); from psql to test it out. If your config is working, you'll see the logged SQL statement that took 1 second in the postgres log file.

```
pid=90062 query_id=440101247839410938: LOG:  \
duration: 1000.963 ms  plan:
        Query Text: select pg_sleep(1);
        Result  (cost=0.00..0.01 rows=1 width=4)
pid=90062 query_id=440101247839410938: LOG:  \
duration: 1001.715 ms  statement: select pg_sleep(1);
```

Follow this approach for your production instance and start by logging very slow queries. This will be a way to begin capturing full queries with parameters using your logs.

Once you have a full query with parameters, you'll want to capture a query execution plan manually by running your query with EXPLAIN.

What if you could automatically capture query execution plans?

Automatically Gathering Execution Plans

The PostgreSQL auto_explain module can be used to automatically capture query execution plans. You wouldn't want to capture every execution plan, but fortunately it can be configured to capture query execution plans for slow queries.

To limit the execution plan capture to slow queries, you'll open up your PostgreSQL config file again and set another parameter. Set the auto_explain.log_min_duration parameter to 1000 as you did earlier for the log_min_duration_statement. This means for queries that take 1 second or more the query execution plan will be logged into the PostgreSQL log file.

For Rideshare, queries run much faster. Try lowering it temporarily to 0 so that all queries have logged execution plans.

If you're tailing the log file with tail -f and the path to your postgresql.log, you'll immediately see the a multi-line logged query execution plan. You'll also see a queryid there, which became more useful in newer versions of PostgreSQL (See: *Using Query ID in Postgres 14*[15]). The logged query execution plan should be the same as the one you generated manually using EXPLAIN.

With the Query Identifier or queryid you can now connect some of the dots between PGSS, logged queries, and execution plans.

1. Identify the queryid from a normalized query in PGSS

15. https://blog.rustprooflabs.com/2021/10/postgres-14-query-id